JSCONF JP 2026

# ワークフローからハーネスへ

Web フレームワークの作者たちが AI エージェントに向かう理由

azukiazusa

# 自己紹介

- **azukiazusa**
- Frontend Engineer
- https://azukiazusa.dev
- 週に 1 回、Web 開発と AI の記事を書いています
- FE（フロントエンド | ファイアーエムブレム）が好き



# Web の作り手たちが、エージェントを作り始めた

- Gatsby のチーム: **Mastra**
- Next.js（Vercel）: **eve**
- Astro（Fred K. Schott）: **Flue**

## Web と AI、一見つながりがなさそうなのに？

この疑問が今日の発表のきっかけとなった

AI エージェントは
いまが抽象化のタイミング

エージェントの課題が
「フレームワークの課題」になった

# エージェントは「フレームワーク以前の Web」

"Agents today are where the web was before frameworks"

- 開発者はみな、同じ土台を手作りしている
- 手作りした土台は、次のプロジェクトに持ち越されない

出典: Vercel, "Introducing eve" (2026) https://vercel.com/blog/introducing-eve

STEP 1

# ワークフローからハーネスへ

# ワークフロー中心: 流れを明示的に組み立てる




- 初期の LLM アプリでは、手順を開発者が明示する設計が有効だった
- モデルに任せる範囲を限定し、**処理の流れ**を組み立てた
- ワークフローは今も、再現性が必要な処理で重要

# ハーネス中心: 抽象化の重心が外側に移った

"Agent = Model + Harness"

- モデルがツール選択や次の行動を判断できる場面が増えた
- **処理の流れだけでなく、実行環境を支える抽象化が重要に**
- 状態・権限・安全性・継続実行を支える足場 = **ハーネス**

出典: Vivek Trivedy, "The Anatomy of an Agent Harness", LangChain Blog

# Mastra：エージェントループだけでは足りない

- Mastra Code の開発から得られた教訓
- 単純なエージェントループでは、長時間の実行に耐えられない
- **コンテキスト管理・永続化・承認**などをハーネスで担う

出典: Sam Bhagwat, "Anatomy of a harness" (2026)

# ハーネスに必要なものが見えてきた

- ツール
- スキル
- サブエージェント
- サンドボックス
- Human-in-the-loop
- 耐久性
- メモリ
- 観測・評価
- チャンネル

共通パターンが見えてきた
↓
フレームワークの出番

STEP 2

# 実績のある Web の設計を持ち込む

# Web の作者が得意なのは「複雑さを引き受ける」こと

WEB フレームワーク

- ルーティング・認証・状態管理
- ビルド・デプロイ・監視
- 規約によって共通処理を隠す

AGENT フレームワーク

- ツール接続・権限・承認
- 永続化・サンドボックス・監視
- 実行基盤の共通処理を隠す

開発者は「どう動かすか」より、**「何を作るか」**に集中できる

# ファイルベースの規約

NEXT.JS

```
app/
├── page.tsx
├── blog/
│   └── page.tsx
└── api/
    └── route.ts
```

EVE

```
agent/
├── agent.ts
├── instructions.md
├── tools/
│   └── get_weather.ts
├── skills/
│   └── plan_a_trip.md
└── channels/
    └── slack.ts
```

"Just as Next.js turns a folder into a route by owning the routing, eve turns a file into an ability by owning the agent loop."

# Flue 2.0: Hooks で状態に応じて能力を組み立てる

```
export function SupportAgent() {
  const [verified, setVerified] =
    usePersistentState("verified", false);

  useTool({ name: "verify", description: "...",
    run: () ⇒ setVerified(true) });

  if (verified) useTool(refundOrder);
  return instructions;
}
```

# 抽象化は、使われてから見直される

- 静的な定義は、複雑なエージェントや複数ステップの処理で限界が見えた
- 破壊的変更をしてでも、直す価値があると判断した
- これは Web フレームワークの世界でも何度も見てきた光景

出典: Flue, "Flue 2.0" https://flueframework.com/blog/flue-2/

STEP 3

# なぜ TypeScript なのか

# TypeScript の活躍する領域が広がっている

モデル・データに近い領域

## Python が強い

- ML・データ処理・実験
- モデルや評価のエコシステム

プロダクトに近い領域

## TypeScript が活きる

- サーバー・ツール・Web UI
- npm 資産・デプロイとの統合

エージェントがプロダクトに近づいた

# エージェントと人をつなぐ UI は、もともと Web の仕事

- エージェントは最終的に、UI を通して人と対話する
  - Slack との接続
  - Web UI への描画
- → UI からサーバー、ツールまで同じ JS/TS 資産を使える

# まとめ

- ワークフローは今も重要。モデルの進化により、**抽象化の重心がハーネスにも広がった**
- Web の作者は、共通の複雑さを引き受ける**開発体験の設計**をしてきた
- エージェントがプロダクトに近づき、**TypeScript の活躍する領域が広がった**

# 参考資料


- Vercel, "Introducing eve" https://vercel.com/blog/introducing-eve
- vercel/eve https://github.com/vercel/eve
- Flue https://flueframework.com
- Flue, "Flue 2.0" https://flueframework.com/blog/flue-2/
- Mastra Docs https://mastra.ai/docs
- Mastra About https://mastra.ai/about
- LangChain, "The Anatomy of an Agent Harness" https://langchain.com/blog/the-anatomy-of-an-agent-harness